

the model’s matrices taken along a direction that has a dimensionality d . Static neurons can also be projected into tokens using LL: Geva et al. (2021), Geva et al. (2022b) observe that neurons of the first MLP matrix FF_1 determine the extent to which each neuron in FF_2 contributes to the intermediate prediction. Dar et al. (2022), Geva et al. (2023) employ the same approach to investigate the attention matrices. Elhage et al. (2021), Katz and Belinkov (2023) demonstrate how these neurons can elucidate model behavior, Wang et al. (2023), Millidge and Black (2022), Todd et al. (2023) use it to explore circuits and in-context learning.

Despite the growing interest in this approach, we are only aware of works that have applied it to the static weights of models or the hidden states of the forward pass. In contrast, our work is focused on the backward pass of LMs.

4 Backward Lens

In this section, we detail the methods we developed to analyze gradients based on our understanding of how each gradient matrix is formed. In the following sections, we consider the editing of a single sequence (prompt) to answer a single token (target) using NLL loss and without employing batching.

4.1 The Rank of the Gradient Matrices

Hu et al. (2022) and Mitchell et al. (2021) have observed the low-rank of MLP layers’ gradients with a single input. However, they did not explain this phenomenon in the context of a matrix with a sequence of inputs, nor did they predict this rank. Building on the observations of Kiani et al. (2022), who bounded the rank of gradients in convolutional and vanilla recurrent networks, we define the following lemma for GPT:

Lemma 4.1. *Given a sequence of inputs of length n , a parametric matrix W and a loss function L , the gradient $\frac{\partial L}{\partial W}$ produced by a backward pass is a matrix with a rank of n or lower.*

Proof. According to Equation 6, the gradient of a matrix is $\frac{\partial L}{\partial W} = x^\top \cdot \delta$. Assuming x, δ are non-zero vectors, the rank of the gradient matrix is 1, given its interpretation as a span, a linear combinations, of a single column vector x , or equivalently, as a span of a single row δ . In the case when x or δ is a zero vector, the rank of the gradient matrix is 0.

In LMs, an input prompt comprises a sequence of n tokens, each of which introduces an intermediate input (x_i) at every layer. In this case, the

gradient matrix is the sum of each x_i, δ_i ’s product:

$$\frac{\partial L}{\partial W} = \sum_{i=1}^n \frac{\partial z_i}{\partial W} \frac{\partial L}{\partial z_i} = \sum_{i=1}^n x_i^\top \cdot \delta_i \quad (8)$$

The maximum rank of the summed gradient matrix is n given each x_i , or δ_i , is linearly independent, since we sum n distinct rank-1 matrices. This rank would be lower than n if there are linear dependencies between x_i or between δ_i , with 0 being the minimum possible rank. For instance, if a linear dependency exists between only two δ_i ’s, the rank of the gradient matrix would be $n - 1$. $\square$

Of particular interest is the case of the last layer of the transformer. In this case, the rank of the gradient is one, see Appendix A.

4.2 Gradient Matrices as Spans of Vectors

In our analysis we focus on the MLP layers, due to recent interest in identifying and editing the knowledge stored in these layers’ neurons (Geva et al., 2022b, 2021; Dai et al., 2022; Mitchell et al., 2021; Meng et al., 2022). Consider the matrices of the MLP modules, FF_1 and FF_2 , each having d_m neurons, which are d -sized vectors. Exploring all the modules’ dimensions is prohibitive, see Appendix C. However, Equation 8 reveals that every gradient matrix is a sum of n outer products $x_i^\top \cdot \delta_i$. This view allows us to examine every gradient matrix as a sum of n pairs of vectors. Therefore, assuming we edit a sequence of inputs with under a few hundred tokens, our analysis would focus on only $n \ll d < d_m$ vectors in $\mathbb{R}^d$.

Every matrix formed by $x_i^\top \cdot \delta_i$ can be interpreted in two ways simultaneously: (1) as a span (linear combination) of x_i and (2) as a span of δ_i . Figure 2 illustrate the two viewpoints. We utilize this duality and examine gradients as the spans of n vectors, x_i or δ_i , guided by the observation that for each module there is only one span with vectors that are in the size of the module’s neurons.

The gradients of FF_1 δ_i is not of size d and thus are not suitable for methods that examine vectors of the hidden state size. However, x_i are d -sized vectors and were previously explored using LL (Geva et al., 2022b; Dar et al., 2022). Therefore, for FF_1 we chose to observe the gradient matrix as a span of x_i , considering δ_i as the factor determining the extent of the update each x_i will introduce (the upper option in Figure 2). Explicitly, we refer to x_i as FF_1 ’s **spanning set** since the j -th neuron of the

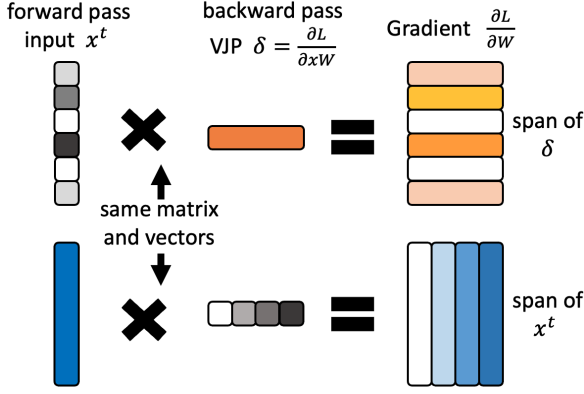


Figure 2: The calculation of gradient matrix by the outer product of $x^\top \cdot \delta$. Each row consists of the same values, but above we describe the matrix as a span (linear combinations) of δ , while below as a span of $x^\top$. The displayed vectors are presented transposed to emphasize the spanning effect.

gradient matrix is equal to a linear combination of x_i :

$$\frac{\partial L}{\partial FF_1}[j] = \sum_{i=1}^n x_i^\top \cdot \delta_i[j], \quad (9)$$

where $\delta_i[j]$ is the j -th element of the vector δ_i .

The gradients of FF_2 The sizes of FF_2 's x_i, δ_i are switched from those of FF_1 , hence we chose δ_i as FF_2 's gradient spanning set. Thus, its gradient's j -th neuron is viewed as a combination of δ_i :

$$\frac{\partial L}{\partial FF_2}[j] = \sum_{i=1}^n \delta_i \cdot x_i[j], \quad (10)$$

where $x_i[j]$ is the j -th element of the vector x_i . In Section 5 we provide a theoretical explanation of why the choice of the VJP δ_i is not only a technical one, due to dimensionality considerations.

Logit Lens Since each spanning set comprises d -sized vectors, we can project them into tokens using LL. In the subsequent sections, we offer a theoretical explanation for why we expect to discover token embeddings in the vectors of these spanning sets and empirical examinations of their LL projections.

5 Understanding the Backward Pass

The VJPs, δ_i , are the hidden states of the backward pass and the vectors that constitute the gradient matrices (Section 3.2). In this section, we try to shed light on what information is encoded in the VJP. Additionally, we aim to explain how Equation 9 and Equation 10 cause the model to change

its internal knowledge. For simplicity, we ignore Dropouts and Layer Norms.

5.1 The VJPs of the Top Layer

In this section, we analyze the initial VJPs that are created during the editing of a single prompt, as we describe in Section 3.2. The last matrix of an LM, which is the final model parameter used before calculating the loss score, is the decoding matrix $D \in \mathbb{R}^{d \times |\text{vocabulary}|}$. During the forward pass, this matrix calculates the output vectors $x_i D = \hat{y}_i$. When editing a prompt, we only use the final prediction of the last prompt's token $x_n D = \hat{y}_n$ to calculate the loss score.

We calculated D 's VJP of the last token δ_n . According to Equation 5, the backward pass' VJP to the layer that preceded D is given by the following backward step:

$$\frac{\partial L}{\partial x_n} = \frac{\partial L}{\partial \hat{y}} D^\top = \delta_n D^\top \in \mathbb{R}^d \quad (11)$$

This result can be simplified as a weighted sum of D 's columns:

$$\delta_n D^\top = \sum_{k=1}^{|\text{vocabulary}|} \delta_n[k] D^\top[k], \quad (12)$$

where $D^\top[k] \in \mathbb{R}^d$ is the k -th column of D . Since D is the decoding (un-embedding) matrix, the k column of D holds the embedding of the model's k -th token. From the equation, $\delta_n[k] \in \mathbb{R}$ controls the magnitude by which we add the embedding of the k -th token into $\frac{\partial L}{\partial x_n}$. Plugging Equation 4 and using the notation $\hat{p} = \text{Softmax}(\hat{y}_n), t$ from Equation 2:

Lemma 5.1. *The VJP $\delta_n D^\top$ passed at the beginning of a backward pass is a vector in $\mathbb{R}^d$ that is a sum of weighted token embeddings. It is dominated by the embedding of the target token, $D^\top[t]$, multiplied by a negative coefficient $\delta_n[t] = \hat{p}[t] - 1$. The embedding of all other tokens $k \neq t$ are scaled by a positive coefficient $\hat{p}[k] D^\top[k]$.*

The VJP $\delta_n D^\top$ is the initial vector to be passed in the backward pass, if we simplify the effect of Layer Norms, which are discussed in Appendix B. In particular, this is the only VJP to span the last MLP layer's gradient.

The LL of δ_n is provided as $\text{Softmax}(\ln_f(\delta_n D^\top))$. Except for $\ln_f$, this behaves similarly to $\text{Softmax}(\delta_n D^\top)$. As the lemma shows, $\delta_n D^\top[t]$ is negative, while $\delta_n D^\top[k]$

is positive for all tokens $k \neq t$. Therefore, we can expect the target embedding to have the lowest probability in the softmax. In practice, since related tokens have more similar embeddings and similar entries in y_n , this effect is expected to be even more pronounced.

5.2 Storing Knowledge in LMs

In Section 4.2 we observed that each neuron in the MLP’s gradients is a sum of vectors in $\mathbb{R}^d$ from the forward and backward passes, x_i and δ_i respectively. Based on this observation, we aim to understand how LM editing with a single prompt and a single backward pass changes the internal knowledge of a model. Explicitly, we study the implications of updating a weight matrix with its gradients: $W \leftarrow W + \eta \frac{\partial L}{\partial W}^\top$, where $\eta \in \mathbb{R}$ is a negative learning rate. In particular, while $\frac{\partial L}{\partial W} = \sum_{i=1}^n x_i^\top \cdot \delta_i$ we are interesting on the direct effect of updating a layer by the gradient of a single token from the edited prompt, $x_i^\top \cdot \delta_i$.

Lemma 5.2. *When updating an MLP layer of an LM using backpropagation only with the VJPs of the i -th token, δ_i , and rerunning the layer with the same inputs x_i from the forward pass of the prompt we used for the editing, the following occurs: (i) The inputs, x_i , are added or subtracted from the neurons of FF_1 , thereby adjusting how much the activations of each corresponding neuron in FF_2 would increase or decrease. (ii) The VJPs δ_i are subtracted from the neurons of FF_2 , amplifying in FF_2 ’s output the presence of the VJPs after they are multiplied with negative coefficients.*

See Appendix D for the proof.

Since the change in FF_1 uses the given inputs x_i to amplify future activation, we term this mechanism “imprint”. The modification of FF_2 is termed as the “shift”, since it represents a process of altering the output of the layer. In summary, the “imprint and shift” mechanism depicts the MLP’s learning process during a single backward pass as having two phases: Given the layer’s original input and the new target, the process imprints a similar input through the update of FF_1 and subsequently shifts the output of FF_2 towards the new target. Figure 1 illustrates this process.

Updating with the full gradient: In practice, the gradient matrix is the sum of individual contributions, each given by the outer product of a token, $x_i^\top \cdot \delta_i$. However, as we will demonstrate in Section 6, only a few tokens dominate the full update.

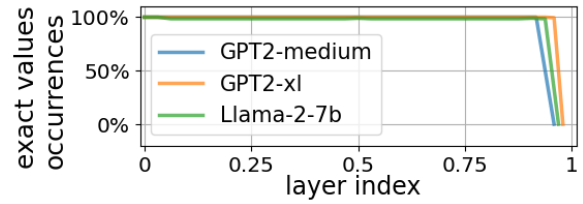


Figure 3: The percentage of occurrences where the rank of FF_1 ’s gradient equals the length of the prompt used for editing. To show different models in the same plot, we normalize the layer indices. Except for the last layer, all layers and models exhibit the above equality more than 98.5% of the time.

Therefore, for the simplicity of our explanation, we can regard the complete update as consisting of just a few individual steps of “imprint and shift”.

LL ranking refers to the index assigned to the vocabulary’s tokens when ordered by the probability scores generated by LL (Equation 7). Updating FF_1 involves adding or subtracting x_i from weights, focusing on the most probable tokens from the LL ranking. Conversely, for FF_2 , updating entails subtracting δ_i , effectively adding $-1 \cdot \delta_i$. This subtraction reverses the LL rankings, turning previously least probable tokens into most probable ones. Thus, when utilizing LL with FF_2 ’s δ_i , attention should be given to the least probable tokens from the projections.

6 Experiments

We conduct a series of experiments to support the results of Sec. 4 and 5, as well as to briefly demonstrate their application to LM analysis.

We employ GPT2 (Radford et al., 2019) and Llama2-7B (Touvron et al., 2023) in our experiments. We randomly sampled 100 prompts and their corresponding editing targets from the CounterFact dataset (Meng et al., 2022). For each model and prompt, we conducted a single backpropagation using SGD and without scaling optimizers, such as Adam (Kingma, 2014), and no batching.

The rank of the gradients To examine Lemma 4.1, we measure the rank of each layer gradient matrix. As depicted in Figure 3, for every prompt with the length of n tokens, the model’s gradient matrices are almost always exactly rank n . The only exceptions are the last MLP layers, which have a rank of 1, as predicted in Section 4. Although unnoticeable from the figure, once in a few dozen examples, there is a drop of one or two